IES NERVIÓN · SEVILLA

SIMULACRO DE EXAMEN · UNIDAD DIDÁCTICA 2

Programación · 1.º DAW · RA3: estructuras de control

Fecha: viernes, 30 de octubre de 2026 | Duración: 2 horas | Lenguaje: Java

Alumno/a: _____ Grupo: _____

INSTRUCCIONES GENERALES

Los seis primeros ejercicios son obligatorios y todos tienen el mismo valor. La calificación se vinculará a los criterios de evaluación indicados en cada ejercicio. El ejercicio 7, redactado en inglés, es voluntario: solo puede subir la nota y nunca restarla.

Crea tú mismo en Eclipse un proyecto Java llamado SimulacroUD2_RA3, con los paquetes ejercicios, test y utiles, tal como se indica en el esquema siguiente. Implementa cada apartado como un método independiente en ejercicios.Ejercicios, respetando exactamente las cabeceras. Puedes crear métodos auxiliares si los necesitas.

Crea la clase TestEjercicios dentro del paquete test e incluye allí el método main, dedicado únicamente a las pruebas. En el paquete utiles crea la clase Utiles y copia de tus proyectos anteriores los métodos esPrimo(Integer) y esPerfecto(Integer), que podrás reutilizar.

Estructura del proyecto en Eclipse



Crea los paquetes y las clases. En TestEjercicios escribe el main.

Se permite consultar los propios proyectos de clase, pero no Internet. No se exige lectura por teclado ni se evalúan las excepciones.

EJERCICIOS OBLIGATORIOS

EJERCICIO 1. La taquilla del cine

Criterios de evaluación RA3: a) estructuras de selección; e) programas ejecutables con estructuras de control.

Un cine cobra 8 € de lunes a jueves, 10 € los viernes y 12 € los sábados y domingos. Los menores de 12 años y las personas de 65 años o más obtienen 3 € de descuento. Los socios obtienen otros 2 € de descuento. Los descuentos son acumulables.

Crea un método que reciba la edad, el día de la semana y si la persona es socia, y devuelva el precio final de la entrada. El día llega en mayúsculas y sin tildes (por ejemplo, MIERCOLES o SABADO). Si la edad es negativa o el día no existe, devuelve -1. Puedes considerar que los argumentos no son null.

Usa obligatoriamente switch para establecer el precio base e if para aplicar los descuentos.

```
public static Integer precioEntrada(Integer edad, String dia, Boolean socio)
```

Ejemplos: precioEntrada(10, "SABADO", true) → 7; precioEntrada(35, "LUNES", false) → 8; precioEntrada(66, "VIERNES", true) → 5; precioEntrada(-1, "LUNES", false) → -1.

EJERCICIO 2. Dibujar una Z con asteriscos

Criterios de evaluación RA3: b) estructuras de repetición; g) comentario y documentación del código.

Escribe un método que reciba un número entero llamado lado y dibuje una letra Z utilizando asteriscos. La primera y la última fila estarán totalmente rellenas de asteriscos. En las filas intermedias debe aparecer un único asterisco, formando la diagonal de derecha a izquierda. Las demás posiciones serán espacios. No hay que pedir datos por teclado.

Para lado = 5, el resultado debe ser exactamente el siguiente:

```
*****
    *
   *
  *
*****
```

Para lado = 1 se imprime un único asterisco. Para lado = 0 no se imprime nada. Puedes suponer que lado nunca es negativo.

Utiliza exactamente dos bucles for anidados y un if-else para decidir si debes imprimir un asterisco o un espacio. Añade comentarios breves que expliquen la finalidad de cada bucle.

```
public static void dibujarZ(Integer lado)
```

EJERCICIO 3. Suma de números especiales

Criterios de evaluación RA3: b) estructuras de repetición; c) sentencias de salto.

Un número es especial si es primo o perfecto. Recibe un intervalo [inicio, fin] y una cantidad. Recorre el intervalo de menor a mayor y devuelve la suma de los primeros 'cantidad' números especiales encontrados. Si un número cumple ambas condiciones, solo se suma una vez.

Puedes utilizar Utiles.esPrimo(Integer) y Utiles.esPerfecto(Integer), que ya están implementados. Utiliza un bucle for con una condición if y termina el recorrido con break cuando hayas encontrado la cantidad solicitada. No utilices continue.

Si inicio es mayor que fin, si cantidad es 0 o negativa, o si no hay números especiales, devuelve 0. Si hay menos números especiales de los solicitados, suma todos los encontrados.

```
public static Integer sumaPrimerosEspeciales(Integer inicio, Integer fin, Integer cantidad)
```

Ejemplos: sumaPrimerosEspeciales(1, 10, 3) → 10 (2 + 3 + 5); sumaPrimerosEspeciales(1, 10, 5) → 23 (2 + 3 + 5 + 6 + 7); sumaPrimerosEspeciales(8, 15, 2) → 24 (11 + 13).

EJERCICIO 4. Factorial con saltos

Criterios de evaluación RA3: e) programas ejecutables con estructuras de control; g) comentario y documentación del código.

El factorial tradicional multiplica un número por todos los enteros positivos anteriores. Vamos a modificarlo: comenzamos multiplicando por numero y, en cada paso, restamos el valor salto hasta llegar a 0 o a un número negativo. Entonces dejamos de multiplicar.

Por ejemplo: factorialConSaltos(7, 2) = 7 × 5 × 3 × 1 = 105; factorialConSaltos(8, 3) = 8 × 5 × 2 = 80; factorialConSaltos(5, 1) = 5 × 4 × 3 × 2 × 1 = 120.

Implementa el cálculo de forma obligatoriamente recursiva, sin utilizar bucles. El caso base debe devolver 1 cuando numero sea menor o igual que 0. Puedes suponer que numero es no negativo y salto es mayor o igual que 1; se usarán valores pequeños para evitar desbordamientos.

Comenta brevemente el caso base y la llamada recursiva.

```
public static Integer factorialConSaltos(Integer numero, Integer salto)
```

Ejemplos: factorialConSaltos(7, 2) → 105; factorialConSaltos(8, 3) → 80; factorialConSaltos(5, 1) → 120; factorialConSaltos(0, 2) → 1.

EJERCICIO 5. Detectar picos de una ruta

Criterios de evaluación RA3: a) estructuras de selección; f) prueba y depuración de programas.

Una lista de enteros almacena las alturas de distintos puntos de una ruta. Una posición intermedia se considera un pico cuando su altura supera a la de ambos vecinos (anterior y siguiente) en, al menos, una diferencia mínima.

Crea un método que devuelva el número de picos. El primer y el último elemento nunca pueden ser picos. Si la lista tiene menos de tres elementos, devuelve 0. Puedes suponer que no hay elementos null y que diferenciaMinima es positiva.

Usa un for clásico y los métodos size() y get() de List<Integer>. No se pide modificar la lista. Para evaluar las pruebas, añade en TestEjercicios al menos dos pruebas propias distintas de los ejemplos; indica en comentarios el resultado esperado y muestra el resultado obtenido.

```
public static Integer contarPicos(List<Integer> alturas, Integer diferenciaMinima)
```

Ejemplos: contarPicos(List.of(2, 5, 1, 6, 3, 9, 7), 2) → 3 (picos: 5, 6 y 9); contarPicos(List.of(2, 5, 4, 7, 4), 3) → 1; contarPicos(List.of(1, 2), 1) → 0.

EJERCICIO 6. Buscar por suma de cifras

Criterios de evaluación RA3: c) sentencias de salto; f) prueba y depuración de programas.

Recibe una lista de números enteros no negativos y una suma objetivo. Recorre la lista de izquierda a derecha y calcula la suma de las cifras decimales de cada número. Devuelve la posición (índice, empezando en 0) del PRIMER número cuya suma de cifras coincida con la suma objetivo.

Para calcular la suma de cifras, utiliza operaciones aritméticas (%, /) y un bucle while; no conviertas los números en cadenas. Utiliza un bucle for para recorrer la lista y, cuando encuentres la primera coincidencia, finaliza la búsqueda obligatoriamente con break. Si no hay coincidencias o la lista está vacía, devuelve -1.

Comprueba y depura el método desde TestEjercicios: añade al menos dos pruebas propias diferentes de los ejemplos, indicando en comentarios la entrada, el resultado esperado y el obtenido. Debes probar un caso con coincidencia y otro sin coincidencia.

```
public static Integer primeraPosicionSumaCifras(List<Integer> numeros, Integer sumaObjetivo)
```

Ejemplos: primeraPosicionSumaCifras(List.of(42, 153, 85, 29), 9) → 1; primeraPosicionSumaCifras(List.of(42, 153, 85, 29), 13) → 2; primeraPosicionSumaCifras(List.of(0, 31), 0) → 0; primeraPosicionSumaCifras(List.of(42, 153), 18) → -1.

OPTIONAL ENGLISH BONUS

EXERCISE 7. Non-overlapping pairs

RA3 assessment criteria: b) repetition structures; f) program testing and debugging.

Write a method that counts how many non-overlapping pairs of adjacent integers have a sum that is a multiple of 5.

Read the list from left to right. If two adjacent numbers form a valid pair, count that pair and skip both numbers. Otherwise, move forward by one position. A number cannot belong to two counted pairs.

Use a while loop. Every local variable name must be in English. You may assume the list contains no null values. An empty list or a list with one element produces 0.

```
public static Integer countNonOverlappingPairs(List<Integer> numbers)
```

Examples: countNonOverlappingPairs(List.of(2, 3, 2, 8, 7)) → 2; countNonOverlappingPairs(List.of(1, 2, 3)) → 1; countNonOverlappingPairs(List.of(1, 1, 1)) → 0.

This optional exercise can only improve your grade; not submitting it or making mistakes in it will never reduce your grade.

ENTREGA

Comprueba tus métodos desde TestEjercicios. Al finalizar, exporta el proyecto Eclipse completo en un único archivo ZIP y entrégalo con el nombre exacto apellidosnombresimulacrotema0.zip (sustituye apellidosnombre por tus apellidos y tu nombre, sin espacios ni tildes). Los ejemplos del enunciado no sustituyen las pruebas propias que se solicitan.